# A Reference Implementation and OMNeT++ Evaluation of a Low-Cost Hybrid ECC Authentication Protocol under Its Stated Environment

Dr. Shanta Sondur
*Project Guide*
*Department of Information Technology*
*Vivekanand Education Society's*
*Institute of Technology*
Mumbai, India
shanta.sondur@ves.ac.in

Jay Kerkar
*Department of Information Technology*
*Vivekanand Education Society's*
*Institute of Technology*
Mumbai, India
2023.jay.kerkar@ves.ac.in

Chaitanya Kelkar
*Department of Information Technology*
*Vivekanand Education Society's*
*Institute of Technology*
Mumbai, India
2023.chaitanya.kelkar@ves.ac.in

Chirag Poornamath
*Department of Information Technology*
*Vivekanand Education Society's*
*Institute of Technology*
Mumbai, India
2023.chirag.poornamath@ves.ac.in

Yash Kolekar
*Department of Information Technology*
*Vivekanand Education Society's*
*Institute of Technology*
Mumbai, India
d2024.yash.kolekar@ves.ac.in

***Abstract*—Al-Rasheed et al. proposed a hybrid elliptic-curve/symmetric-key mutual-authentication protocol for IoT devices and evaluated it in OMNeT++ without releasing code. Since neither code nor simulation model is available, we ask what can be established independently. We provide an open reference implementation, a C port of its device side that runs on an ESP8266, and independent reference vectors that both implementations reproduce byte for byte while rejecting tampered and replayed messages. Checking the paper against itself, we find that its tables disagree with its text (four messages and 1024 bits against two messages and 264 bits), that its headline savings of about 17%, 21% and 12% cannot be derived from its Tables IV–VI, and that its constants and comparison rows also appear in an earlier paper. We then model the stated environment (its Table II network and Table III per-operation times) in OMNeT++: the model reproduces the paper's per-session cost of $2T_H + 2T_{SE/D}$ per side (23.68 ms by the paper's arithmetic, 23.98 ms simulated) and the sizes of its Table IV. Timing the primitives on the ESP8266 and re-running the model lowers the cost to 2.25 ms, and the server-side pseudonym lookup, which the paper leaves open, decides scalability: flat with a hash index, growing with the number of devices under a linear scan. The paper's simulated figures cannot be checked without the authors' model, which we have requested. The implementation, model, raw hardware logs and scripts are open.**

***Index Terms*—IoT authentication, elliptic curve cryptography, OMNeT++, ESP8266, reproducibility**

## I. Introduction

Lightweight mutual authentication for resource-constrained devices is commonly evaluated by counting cryptographic operations and weighting them with per-operation times. Al-Rasheed et al. [1] propose a protocol in which devices and servers share long-term secrets from an offline phase and authenticate online with symmetric primitives only, and they evaluate it in OMNeT++ against other schemes. No implementation or simulation project accompanies the paper.

This paper contributes (i) an open reference implementation of the protocol (Python, AES-GCM, HKDF-SHA256, SHA-256) and a portable C device core that runs on an ESP8266 and interoperates with the Python server; (ii) independent reference vectors, computed without any of the implementation's code, that both implementations reproduce byte for byte, together with an explicit register of the decisions the paper leaves open; (iii) a consistency check of the paper's tables against its own text and references, which does not depend on our implementation; (iv) an OMNeT++ model of the paper's stated environment, parameterised only by its Tables II–V, and a check that implementation and model reproduce its per-session cost and communication sizes; (v) hardware timings of the primitives behind the paper's cost constants, and the same model re-run with them; and (vi) a scalability study of the server-side pseudonym lookup, which the paper does not specify.

## II. Protocol and Implementation

We implement the nonce-based exchange of Section III of [1]. A trusted authority (TA) derives each entity's long-term secret $\lambda_X = H_1(ID_X \,\|\, s)$ locally and distributes pairwise keys $k_{ij}^{\star}$ in an offline phase. In an epoch $t$ a device $V_i$ derives

a pseudonym $PID_{V_i,t} = H_2(\lambda_{V_i} \,\|\, t)$, samples a nonce $N_i$, and sends

$$M_1 = \langle PID_{V_i,t},\, C_1,\, \mu_1 \rangle,$$
$$(C_1, \mu_1) = \mathrm{AEAD}_{k^\star_{ij}}(PID_{V_i,t} \,\|\, N_i \,\|\, t).$$

The server resolves the pseudonym to a known device and epoch, verifies and decrypts $M_1$, samples $N_j$, answers with $M_2$ (the same construction), and both sides derive $K_{ij,t} = \mathrm{KDF}(k^\star_{ij} \,\|\, N_i \,\|\, N_j \,\|\, t)$. Replayed messages are rejected by an epoch window per peer. We instantiate AEAD as AES-256-GCM [7], KDF as HKDF-SHA256 [5] and $H$ as SHA-256, as suggested in the instantiation notes of [1]. The wire format is a length-prefixed JSON/base64 envelope.

The device side is also written as a portable C core on BearSSL [4] with the identical wire format. It runs natively against the unmodified Python server (12 of 12 handshakes) and on a NodeMCU ESP8266 over Wi-Fi (20 of 20 handshakes). In every handshake the session-key fingerprints on the two sides were identical and a replayed $M_1$ was rejected.

## III. Conformance to the Specification

### A. Specification and assumptions

The paper fixes the equations but not their encodings. We therefore froze one specification of Section III of [1] (Eqs. 8–24) and classified each decision as stated by the paper, suggested by its instantiation notes, or our assumption. The assumptions cover byte encodings, HKDF parameters and domain-separation strings, the GCM IV and associated-data use, the wire format and epoch handling. The register, and a matrix from each equation to its Python and C code and to the evidence that covers it, are part of the repository. The C port implements the device half only; coverage is an abstract predicate and the optional rekeying of Eq. 24 is a stub.

### B. Independent reference vectors

A generator that imports nothing from the implementation restates the specification using the Python standard library and a direct AES-256-GCM call with chosen IVs. It first checks its own primitives against RFC 5869 (cases A.1 and A.3) [5], NIST GCM test case 16 [7] and the FIPS 180 SHA-256 vector for "abc" [6]. It then fixes the identities, master secret, epochs, protocol nonces and GCM IVs for two epochs, and computes the long-term secrets, pairwise root, pseudonym, plaintexts, ciphertexts, tags, wire bytes and session key. The implementations must reproduce every value (Table I).

Deliberately corrupting a vector makes both suites fail, so the comparison can detect a wrong value. A dash marks a server-side check that the C core does not implement.

### C. What this does and does not show

The vectors show that both implementations agree with an independent restatement of the specification, so a mistake in code they share would be caught. They do not show that the specification is the authors': five questions remain that only they can answer (the associated data covered by $\mu_1$, the construction of $\sigma_2$, how epochs advance and persist, how a server resolves a pseudonym, and what enrolment distributes). Nor are they a security proof.

TABLE I
Conformance to the reference vectors (two epochs).

| Property | Python | C device core |
|---|---|---|
| Pairwise root $k^\star$ | match | match |
| Pseudonym, $M_1$ wire bytes | match | match |
| $M_2$ wire bytes, session key $K$ | match | $K$ match |
| Tampered $M_1$ field | rejected | — |
| Tampered $M_2$ field | rejected | rejected |
| Replayed $M_1$ | rejected | — |
| $M_2$ of another session | rejected | rejected |
| Tests / checks | 15 | 15 |

## IV. Evaluation Setup

### A. Environment and constants

The OMNeT++ model uses the parameters of Table II of [1]: a $1500 \times 1500$ m area, 100 to 1000 devices, servers at 5% of the device count, a 10 Mbps channel, 75.6 mW transmit and receive power, a 15 $\mu$s time quantum, and a 500 m range. Operation times are those of its Table III ("reference values for the processing time"): $T_H = 0.32$ ms and $T_{SE/D} = 5.6$ ms. Operation counts and message sizes come from its Tables IV and V, and no parameter is fitted to any result. These tables are not original measurements of the proposed protocol: the five constants of its Table III, the nine competitor rows of its Table IV and the finite entries of its Table VI also appear, with the same numbers, in Tables 3, 5 and 6 of Sadhukhan et al. [2], which it cites as [18]. In [2] the storage columns are headed smart card/device and sensor node, whereas [1] labels the same numbers server and device storage, and an arithmetic slip in [2] (32 + 320 bits stated as 356 instead of 352) is carried over. We observe the correspondence and draw no conclusion about how the values were obtained. The model is written in plain OMNeT++ [3] 6.4.0 (no INET, no SUMO) and is our own implementation of the stated environment.

### B. Assumptions the paper does not fix

The paper leaves several inputs open, and we state ours: an adversary share of 5% of the devices, each sending forged requests at a mean rate of 1 Hz; an authentication window of 10 s in which every device authenticates once at a uniform time; every server resolves every device in its range; and a first-come-first-served server CPU. Devices with no server in range cannot authenticate and are counted separately. Each result is the mean of five seeds. The assumption most relevant to the protocol is how a server resolves a pseudonym (Section V-F).

### C. Hardware testbed

Primitive timings were taken on a NodeMCU ESP8266 (Tensilica L106, stock 80 MHz and 160 MHz, about 80 KB RAM, no cryptographic hardware, Wi-Fi off) with BearSSL, timed by the cycle counter; each figure is the median of 100 runs (24 for elliptic-curve operations). Every benchmark

TABLE II
OPERATIONS AND SIZES PER HANDSHAKE. TABLE V OF [1] LISTS $2T_H + 2T_{SE/D}$ PER SIDE.

| | Device | Server |
|---|---|---|
| AEAD encrypt / decrypt ($T_{SE/D}$) | 1 / 1 | 1 / 1 |
| Pseudonym hash $H_2$ | 1 | 1 per candidate |
| Root KDF, Eq. 9 (first contact) | 1 | 1 |
| Session-key KDF, Eq. 16 | 1 | 1 |
| Candidates scanned (linear) | — | about 8 per device |
| Fields: $M_1$ 116 B, $M_2$ 57 B (1384 bits) | | |
| Encoded (JSON/base64): $M_1$ 216 B, $M_2$ 133 B (2792 bits) | | |

output (hash, AES block, CTR, GCM ciphertext and tag, curve points) was checksummed, and all 35 digests are identical on both clocks and on a laptop running the same code. The whole benchmark was repeated on a second ESP8266 board: the 35 medians agree with the first board and with the earlier run to the microsecond.

## V. RESULTS

### A. Operation profile of the implementation

Table II lists what one handshake of the implementation executes (counted by instrumenting the primitives; the server was enrolled with 1, 20 and 100 devices, the target device last).

The symmetric operations match Table V. Table V does not list the session-key derivation required by the protocol's own key-derivation step or the pseudonym resolution; we report them separately so that the paper's accounting and the implementation's can be compared directly. The table counts a first handshake; once the pairwise root is cached, each side performs only the session-key KDF. Field sizes are PID 32 B, GCM IV 12 B, ciphertext 56 B and tag 16 B for $M_1$, and a 5-byte server identity, IV 12 B, ciphertext 24 B and tag 16 B for $M_2$ (so $M_2$ grows with the identity length); "on the wire" adds the JSON and base64 encoding but no transport framing. Message size depends on the field encodings and the framing, which the paper does not fix; Table IV lists 1024 bits over four messages, and we simulate that figure and the 264-bit, two-message figure of its text. Our field-level count of 1384 bits is above both, and with a 256-bit SHA-256 pseudonym $M_1$ alone exceeds 264 bits.

### B. Communication cost

Fig. 1 shows the bits per session in the simulation without adversaries for the nine schemes of Table IV that have operation counts in Table V. All equal the paper's values, and the proposed scheme has the lowest communication cost. Because Table IV's sizes are inputs to the model, this confirms the pipeline rather than the sizes themselves.

### C. Per-session cost under the paper's constants

With Table III, Table V gives $2 \cdot 0.32 + 2 \cdot 5.6 = 11.84$ ms per side, 23.68 ms per session. Table III compares this with

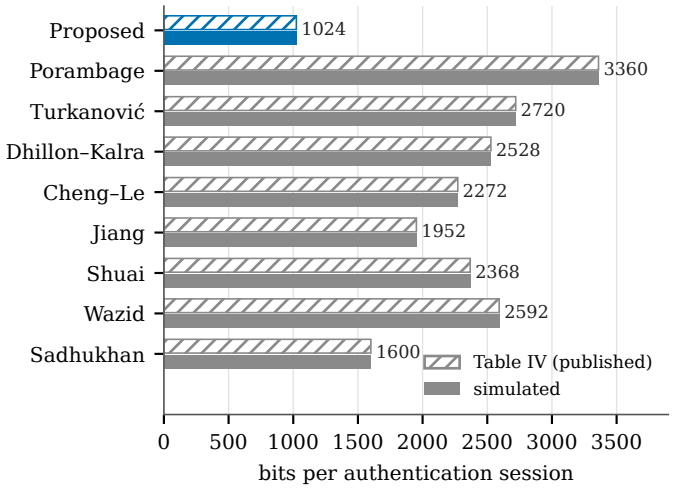



Fig. 1. Bits per authentication session: Table IV of [1] (hatched) and the simulation without adversaries (solid).

TABLE III
PER-SESSION PROCESSING COST (MS), $N = 100$, NO ADVERSARIES, HASH-INDEXED LOOKUP, MEAN OF FIVE SEEDS. "ARITHMETIC" IS TABLE V COUNTS × PER-OPERATION TIMES.

| | Table III constants | | ESP8266, 80 MHz | |
|---|---|---|---|---|
| Configuration | Arith. | Simulated | Arith. | Simulated |
| Table V, 4 msgs | 23.68 | 23.98 | 2.08 | 2.25 |
| Table V, 2 msgs | 23.68 | 23.85 | 2.08 | 2.13 |
| Table V + session KDF | — | 35.18 | 3.34 | 3.51 |

the simulation (no adversaries, hash-indexed lookup). The simulated latency is slightly above the arithmetic because it also contains airtime, rounding to the 15 $\mu$s time quantum and occasional queueing at a server. Adding the session-key derivation, priced at a measured ratio to a hash since Table III has no KDF constant, gives 35.18 ms.

### D. Hardware measurements

Table IV and Fig. 2 give the measured cost of the primitives behind the three constants of Table III. Table III does not name the platform its values correspond to (the same values appear in [2], which names SHA-1 for the hash and attributes the timings to its own references), so we report how they compare with one concrete constrained device. The ratios between operations differ from Table III's: on the ESP8266 a symmetric operation costs about 13 hashes ($T_{SE/D}/T_H$ for the fast AES) and a curve multiplication about 7,600, against 17.5 and 53 in Table III. The AES-GCM figure depends strongly on the AES implementation (0.48 ms table-based, 1.15 ms constant-time), which is a design choice a deployment must make.

**End-to-end on the device.** In the firmware, a handshake costs 1.67 ms to build $M_1$ and 3.59 ms to verify $M_2$ and derive the key, 5.26 ms of device compute in total (mean of 20 handshakes), and 52 ms end to end over Wi-Fi; $M_1$ is 216 bytes. The firmware uses the constant-time AES and includes JSON/base64 parsing, the per-message key schedule and the

TABLE IV
MEASURED OPERATION TIMES ON THE ESP8266 (MS, MEDIAN OF 100 RUNS; 24 FOR CURVE OPERATIONS) AGAINST TABLE III OF [1]. AES-GCM: AES-256, 56 B INCLUDING KEY SETUP, FOR THREE BEARSSL AES IMPLEMENTATIONS. RATIO: 80 MHZ TIME OVER TABLE III.

| Operation | Tab. III | 80 MHz | 160 MHz | ratio |
|---|---|---|---|---|
| $T_H$, SHA-1 (32 B) | 0.32 | 0.0363 | 0.0182 | 0.11 |
| SHA-256 (40 B) | — | 0.0746 | 0.0373 | — |
| $T_{SE/D}$, AES-GCM (fast) | 5.6 | 0.485 | 0.244 | 0.09 |
| small-table | 5.6 | 0.834 | 0.419 | 0.15 |
| constant-time | 5.6 | 1.146 | 0.574 | 0.20 |
| HKDF-SHA256 | — | 0.629 | 0.315 | — |
| $T_{ECM}$, P-256 | 17.1 | 276.4 | 138.2 | 16.2 |
| X25519 | — | 130.9 | 65.4 | — |

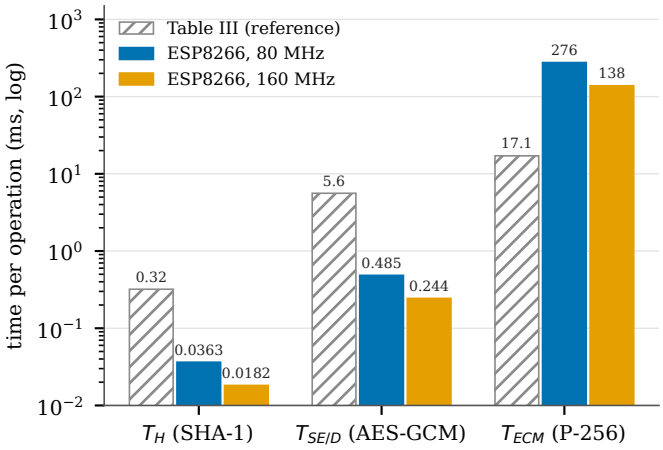



Fig. 2. Cost of the three primitives of Table III: the reference values of [1] (hatched) and the ESP8266 at 80 and 160 MHz (log scale). AES-GCM is the fast table implementation.

session KDF, so it exceeds the operation-count figure for the same constants (Fig. 3).

### E. Simulation with measured constants

Re-running the model with the ESP8266 constants (Table III, right) puts the per-session cost of the proposed scheme at 2.25 ms with Table V's counts and 3.51 ms with the session KDF, against 23.98 ms and 35.18 ms with Table III. Communication cost is unchanged because it does not depend on processing times. The model structure is the same in both runs, and only the per-operation times differ.

### F. Scalability and the pseudonym lookup

The paper does not say how a server maps an incoming pseudonym to a device. We simulate both natural choices: a linear scan over enrolled devices and epochs, as in our implementation, and a hash-indexed table, which costs storage proportional to the number of devices and epochs. Fig. 4 and Table V show the mean per-session latency with the adversary share above, under both sets of constants.

With the hash index the latency is flat at the cost-model value. With a linear scan, latency grows with the number of

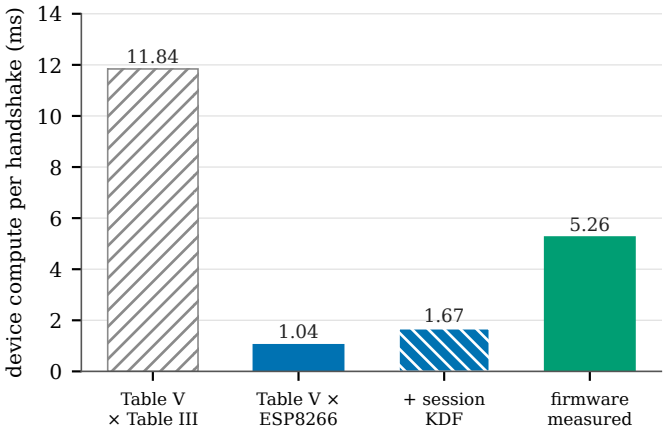



Fig. 3. Device-side compute per handshake. Hatched: Table V counts × Table III. Blue: the same counts × the measured ESP8266 constants (fast AES), without and with the session KDF. Green: measured end to end in the firmware (constant-time AES, parsing included).

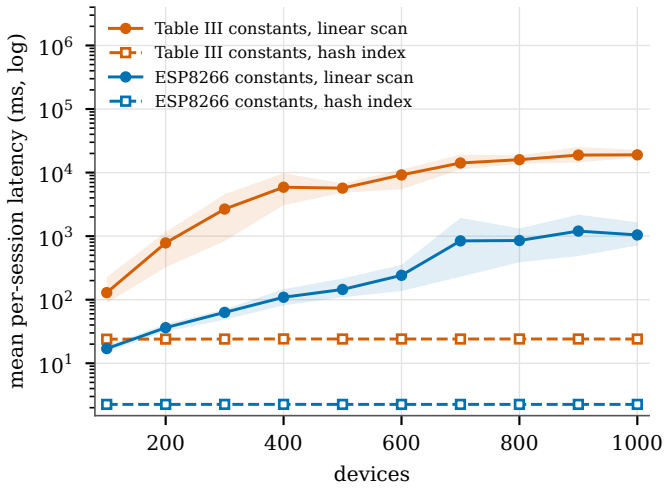



Fig. 4. Mean per-session latency of the proposed scheme against the number of devices (log scale; band: minimum to maximum over five seeds). Table V counts; adversary share 5%.

devices a server must resolve, and the spread across seeds is large near saturation (standard deviation 2141 ms at 1000 devices with Table III constants, 370 ms with the ESP8266 constants). Faster primitives lower the curve by an order of magnitude but do not change its shape. On a laptop-class server our Python implementation resolves a pseudonym in 4.8 ms at 1,000 devices and 23.9 ms at 5,000 (13-epoch window, worst case: device enrolled last); a precomputed table avoids the scan at the price of $N \times W \times 32$ B, 416 KB at 1,000 devices. The choice is therefore one a deployment must make explicitly, trading server time for storage.

### G. Consistency of the published tables

Several checks use only the paper's own tables, text and reference list, not our implementation (Table VI).

TABLE V
MEAN PER-SESSION LATENCY (MS), MEAN OF FIVE SEEDS.

| | Table III constants | | ESP8266 constants | |
|---|---|---|---|---|
| Devices | Linear | Hash | Linear | Hash |
| 100 | 129.2 | 23.93 | 17.0 | 2.25 |
| 300 | 2666.2 | 24.04 | 63.2 | 2.25 |
| 500 | 5691.2 | 24.07 | 145.0 | 2.25 |
| 700 | 14155.8 | 24.08 | 842.7 | 2.25 |
| 1000 | 19009.8 | 24.07 | 1044.6 | 2.25 |

TABLE VI
CONSISTENCY CHECKS ON [1].

| Item | Observation |
|---|---|
| Size | Table IV: 4 messages, 1024 bits. Section V-B: 2 messages, 264 bits, from "128 bits" per message; $2 \times 128 = 256$. |
| Savings | Against its Tables IV–VI the proposed scheme is 36.0% below the best and 57.5% below the mean communication cost, and 33.3% (server) and 10.1% (device) below the best storage. No definition we tried (best, mean, median, worst, each scheme) gives the abstract's 17%, 21% or 12%. |
| Processing | With Table III constants, Table V gives 23.68 ms for the proposed scheme, behind three hash-only rows (3.84, 4.48 and 4.80 ms for device and server). |
| Table V and Eq. 22 | Table V charges $2T_H + 2T_{SE/D}$ per side, whereas Eq. 22 requires one KDF, one encryption and one MAC. A measured HKDF costs 6–17 hashes, which moves the proposed scheme further behind the hash-only rows (about 4–14× on the ESP8266). |
| Labels | The rows "Cheng–Le [14]", "Jiang [15]" and "Shuai [16]" cite papers that are not the schemes named. The Turkanović row swaps the device and server hash counts relative to the original (7/7/5 against 7/5/7). |

These observations rest on the paper alone, apart from the measured HKDF cost in the fourth row. The processing comparison is only as good as the operation counts of the hash-only rows: the Turkanović and Dhillon–Kalra counts were checked against the original papers, whereas the row labelled "Jiang" could not be tied to a source. Figs. 1–3 of [1] depend on the authors' simulation and are not evaluated here; we extracted their plotted values exactly from the PDF's vector graphics, but cannot say how they were produced.

### H. Summary against the published values

Table VII collects the quantities of [1] that can be derived from its stated tables and environment, and what we obtain.

### I. Reproducibility

The model's 3,385-run main sweep was run on two toolchains (OMNeT++ 6.0.3 on Linux and 6.4.0 on macOS arm64); all rows agree to a relative difference of $5 \times 10^{-14}$. The 495 hash-index and uncontended runs were added on macOS only. The hardware benchmark is archived with raw serial logs for both boards and both clocks, and the C device core interoperates with the Python server on the same wire format. Scripts, configuration generators, result tables and the figure script are open, as is the reference-vector generator; everything is in the project repository (https://github.com/jaykerkar0405/hybrid-ecc-iot-auth).

TABLE VII
PUBLISHED VALUES AND OURS. ROWS 1–3 ARE CONSISTENCY CHECKS: THEIR INPUTS ARE THE PAPER'S OWN TABLES.

| Quantity | Published | Ours |
|---|---|---|
| Bits per session, Table IV, nine schemes | 1024 (proposed) to 3360 | identical in all nine |
| Message size, proposed | 1024 bits (Table IV); 264 bits (text) | 1384 bits of fields; 2792 bits encoded |
| Savings, abstract | about 17%, 21%, 12% | not derivable from Tables IV–VI |
| Per-session processing, proposed, Table V × III | 23.68 ms | 23.98 ms (simulated); 23.85 ms (2 msgs) |
| Same counts, ESP8266 constants | not given | 2.08 ms (arithmetic); 2.25 ms (simulated) |
| Device compute, firmware | not given | 5.26 ms (20 runs) |
| Lookup, 1000 devices (Table III constants) | not given | 24 ms (hash index); 19,010 ms (linear) |

## VI. LIMITATIONS

The model is ours, not the authors'; the assumptions of Section IV-B are not specified by the paper and results at saturation depend on them. The results of Sections V-C, V-E and V-F are model output, not measurements, and those under Table III constants use the paper's assumed values. Conformance (Section III) is to our reading of the specification, which only the authors can confirm. Hardware timings come from one microcontroller family with software cryptography and one library; other devices, libraries or hardware AES will differ, and 8- and 16-bit parts were not measured. The end-to-end firmware figures are means over 20 handshakes recorded in our project repository; the per-primitive benchmarks, unlike them, are archived with raw logs. Constants without a measured counterpart ($T_C$, $T_{fe}$) are taken from [1], which reuses those of [2]. The other schemes of its tables are not re-implemented, and their operation counts are taken from the paper, checked against the original papers where we could obtain them. We did not evaluate the figures of [1], whose underlying data is not published. We emailed its corresponding author on 27 October 2026 to request the simulation model and the source of the Table III constants; no reply had arrived by 30 October 2026, and anything that depends on their simulation remains unverified.

## VII. Conclusion

We provide an open implementation, an ESP8266 port and an OMNeT++ model of the protocol of [1], and independent reference vectors that both implementations reproduce. Checking the paper against itself shows that its tables contradict its text and that its headline savings cannot be derived from them. Under the paper's stated environment and constants, our model reproduces its per-session processing cost and communication sizes. Timing the primitives on a constrained device and re-running the model with the measured constants lowers the per-session cost by an order of magnitude, and the server-side pseudonym resolution, which the paper leaves open, determines how the protocol scales with the number of devices. Confirming the remaining points requires the authors' simulation model and their answers to the open specification questions.

## References


[1] A. Al-Rasheed, R. Khan, A. Ahmad, A. Saeed, F. Alturise and S. Alkhalaf, "A low-cost hybrid elliptic curve cryptography authentication protocol for trustworthy Internet of Things communication," *IEEE Trans. Consum. Electron.*, vol. 72, no. 2, pp. 4483–4491, May 2026, doi:10.1109/TCE.2026.3662053.

[2] D. Sadhukhan, S. Ray, G. P. Biswas, M. K. Khan and M. Dasgupta, "A lightweight remote user authentication scheme for IoT communication using elliptic curve cryptography," *J. Supercomput.*, vol. 77, no. 2, pp. 1114–1151, Feb. 2021.

[3] A. Varga and R. Hornig, "An overview of the OMNeT++ simulation environment," in *Proc. SIMUTools*, 2008.

[4] T. Pornin, "BearSSL, a smaller SSL/TLS library," [Online]. Available: https://bearssl.org

[5] H. Krawczyk and P. Eronen, "HMAC-based extract-and-expand key derivation function (HKDF)," RFC 5869, 2010.

[6] National Institute of Standards and Technology, "Secure hash standard (SHS)," FIPS PUB 180-4, 2015.

[7] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM)," NIST SP 800-38D, 2007.